

SMC Reset Unit — Functional Specification

SMC Reset Unit — Specification (External Contract)

1. Role
2. Theory of operation
 - 2.1 The reset hierarchy
 - 2.2 Per-subsystem distribution
 - 2.3 JTAG override plane
 - 2.4 The FLR “cool reset” sequence
 - 2.5 Abstraction in the LT model
 - 2.6 The four derived resets in detail
3. Address window
4. Register map (from `reset_unit.rdl`)
5. Reset-derivation contract
6. JTAG overrides (`set_jtag_ctrl`)
7. Per-subsystem reset-control bundle (`reset_ctrl_t`)
8. FLR cool-reset flow
9. Reset domains for register state
10. TLM-2.0 error taxonomy
11. Timing

SMC Reset Unit — Specification (External Contract)

Status: Modelled · **Style:** SystemC / TLM-2.0 Loosely-Timed (LT) · **CCI:** 1.0

This document is the **external, RTL-traceable contract** of the SMC Reset Unit SystemC model. It describes *what* the model does (register map, observable behaviour, error responses) without prescribing *how* — see `02_RESET_UNIT_LowLevel_Design.md` for the internal design.

1. Role

The Reset Unit is the central reset controller of the System Management Controller (SMC). It:

1. Derives the chip-level **cold**, **primary**, **core**, and **WDT** resets from the pad/power inputs, watchdog timers, fuse-sensing status, and the incoming cool reset.
2. Distributes each reset, synchronised, to the **reference**, **SMC**, and **peripheral** clock domains.
3. Drives a **per-subsystem reset-control bundle** (`reset_ctrl_t`) for each of the 32 SMC subsystems, with software programming and JTAG override.
4. Implements the PCIe **Function-Level-Reset (FLR)** “cool reset” flow: isolate-request generation plus a timed `rst_cool_no` pulse to downstream chiplets.
5. Exposes a **32-bit AXI-Lite register block** to firmware.

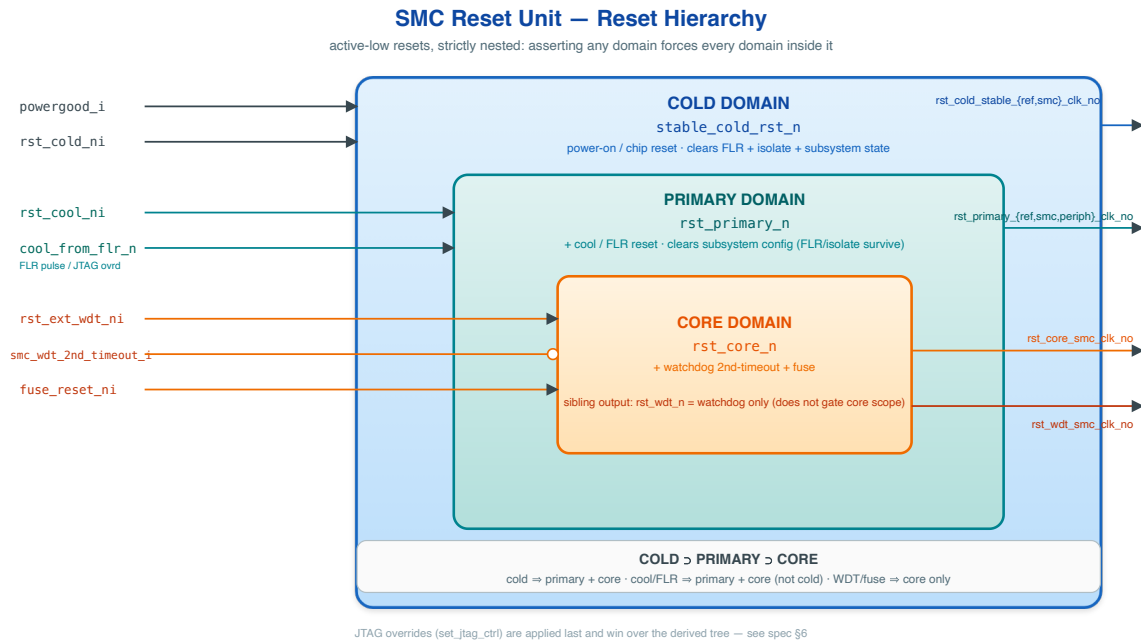
RTL references: `smc_reset_unit.sv`, `smc_reset_ctrl.sv`, `smc_subsystem_resets.sv`, `smc_cool_reset_wrap.sv`, `smc_reset_sync.sv`, `reset_unit.rdl`, `smc_pkg.sv`.

2. Theory of operation

The Reset Unit sits at the root of the SMC reset tree. It consumes the raw pad/power, watchdog, fuse, and cool-reset inputs, folds in firmware programming and JTAG overrides, and produces a **hierarchy of derived resets** that it distributes — synchronised — to every clock domain and to each of the 32 subsystems. Everything the block does can be understood as four cooperating mechanisms.

2.1 The reset hierarchy

Resets are **active-low** and strictly **nested**, strongest first (`cold > primary > core`):



SMC Reset Unit reset hierarchy: three concentric reset domains. The outermost COLD domain (`stable_cold_rst_n`) is driven by `powergood_i` and `rst_cold_ni` and represents power-on / chip reset. Inside it the PRIMARY domain (`rst_primary_n`) additionally folds in `rst_cool_ni` and the `cool_from_flr_n` FLR pulse. The innermost CORE domain (`rst_core_n`) additionally folds in the watchdog second-timeout and `fuse_reset_ni`; `rst_wdt_n` is a sibling output gated by the watchdog only. Each domain drives synchronised reset outputs to the reference, SMC, and peripheral clock domains. Asserting an outer domain forces every domain nested inside it: cold implies primary and core; cool/FLR implies primary and core but not cold; watchdog/fuse gate core only.

A reset asserted at any level implies every level below it: a **cold** reset forces primary and core; a **cool / FLR** reset forces primary and core but *not* cold; a **watchdog or fuse** event gates only core. This nesting is the single most important property of the block — it is what lets firmware state that survives a warm/cool reset (FLR and isolate programming, held in the **cold** domain) coexist with state that must be wiped on every primary reset (subsystem configuration, held in the **primary** domain). §9 makes the domain membership explicit; the boolean equations are in §5.

2.2 Per-subsystem distribution

Beyond the chip-level tree, the unit drives an independent **reset-control bundle** (`reset_ctrl_t`, §7) to each subsystem. Firmware programs per-subsystem cold/warm reset, clock-during-reset forcing, and four “hold” qualifiers (config / SRAM / critical / debug) that tell a subsystem which state to preserve across a warm reset. Each bundle is therefore the join of (a) the chip-level derived resets, (b) the per-subsystem register bits, and (c) any JTAG override. Lock registers (`SS_CONFIG_LOCK`, `SS_COLD_RESET_LOCK`, both **woset**) let firmware make selected bits sticky so later writes — including buggy ones — cannot disturb a subsystem that has already been brought up.

2.3 JTAG override plane

In parallel with the firmware-programmed path, a debug agent can force any of the chip-level resets or any per-subsystem cold/warm reset to a chosen value (§6). Overrides are evaluated *last*, so they win over both the derived tree and the register file. This mirrors `smc_pkg::jtag_smc_reset_ctrl_t` and lets a bring-up engineer

hold or release individual resets independently of firmware.

2.4 The FLR “cool reset” sequence

PCIe Function-Level-Reset is the one genuinely *sequenced* behaviour in the block. A rising edge on `cfg_flr_pf_active_i` latches an isolate request (`ISOLATE_REQ_SMC_REG.bit0`, HW-set / SW-clear), which immediately raises the isolate and mem-repair-skip outputs. After a programmable delay the unit then emits a timed `rst_cool_no` pulse to the downstream chiplets — assert after `ISOLATE_REQ_FLR_COUNTER_VALUE` reference-clock periods, release after a further `ISOLATE_REQ_FLR_RESET_COUNTER_VALUE` periods (a zero reset-count suppresses the pulse entirely). Because the cool reset feeds `rst_primary_n` but not the cold domain, the isolate/FLR programming that initiated the flow deliberately survives it (§2.1, §9). The full contract is in §8.

2.5 Abstraction in the LT model

The model is **loosely-timed**. Reset *derivation* is purely combinational (zero-delay): the RTL’s CDC synchronisers, de-glitch shift registers, and reset extenders/stretchers are collapsed away because they change only the edge timing, not the firmware-observable reset relationships. The two places where time is modelled are (a) the register-access delay annotated on every `b_transport` (`access_delay_ns`) and (b) the FLR pulse, which is paced in real `ref_clk_period_ns` units so its assert/release timing matches the counters above. §11 collects the timing knobs.

2.6 The four derived resets in detail

The block produces four firmware-visible resets. Three of them — **cold**, **primary**, and **core** — are strictly nested (`cold ⊃ primary ⊃ core`); the fourth — **WDT** — is a *sibling* output that is not part of the nested tree but whose generating term also gates core. All four are **active-low** (a *low* level means “in reset”). The boolean equations are in §5; the register-clear consequences are in §9. This section explains *what each reset means*, *what asserts it*, and *what survives it*.

2.6.1 Cold reset — power-on / chip reset (strongest)

`stable_cold_rst_n = powergood_stable & cold_reset_n` where `powergood_stable = powergood_i`,
`cold_reset_n = rst_cold_ni` (JTAG-overridable via `cold_reset_n_ovrd`).

Cold reset is the **root of the reset tree** and models a full power-on or chip-level reset. It is asserted whenever **power is not yet good** (`powergood_i` low) or the **cold-reset pad is asserted** (`rst_cold_ni` low), or when a JTAG agent forces it.

- **Scope / meaning:** everything in the SMC is held in reset. Because cold sits at the top of the hierarchy, asserting it forces **primary** and **core** as well (see §5).
- **State affected:** cold reset clears **both** reset domains — the **cold-domain** registers (the FLR / isolate programming and the FLR counters, §9) *and*, by implying primary, the **primary-domain** register group. A cold reset therefore wipes essentially all firmware-programmed state; nothing in this block is designed to survive it.
- **What it protects:** the cold domain is the *only* state that survives a warm/cool reset. Firmware places FLR / isolate programming there precisely so that it persists across a cool reset but is still cleared on a true power-on.
- **Outputs driven:** `powergood_stable_o`, `rst_cold_stable_ref_clk_no`, `rst_cold_stable_smc_clk_no`.

- **RTL vs. model:** the RTL de-glitches cold reset over 32 ref-clk cycles, extends it 255 cycles after de-assertion, and stretches power-good by 32 cycles. These are pure edge-timing details and are abstracted to zero latency in the LT model (§2.5); the combinational relationship is exact.

2.6.2 Primary reset — SMC “warm” reset (middle)

`rst_primary_n` = `stable_cold_rst_n` & `stable_cool_rst_n` & `cool_from_flr_n` where `stable_cool_rst_n` = `rst_cool_ni`, `cool_from_flr_n` is the FLR-generated cool pulse (JTAG-overrideable via `cool_reset_n_ovrd`).

Primary reset is the SMC’s **warm reset**. It is asserted by **any** of:

1. a **cold** reset (cold implies primary), **or**
 2. an **incoming cool reset** on the `rst_cool_ni` pin (from the primary chiplet), **or**
 3. the locally **FLR-generated cool pulse** (`cool_from_flr_n`, §2.4 / §8).
- **Scope / meaning:** this is the reset of the **SMC register block and per-subsystem configuration**. `rst_primary_smc_clk_no` is the reset that clears the **primary-domain** register group — `SS_CONFIG`, `SS*_HOLD`, the config/cold-reset locks, `SYNC_REG`, etc., with `SS_WARM_RESET_N` reloading to `0xFFFFFFFF` (§9).
 - **Key property — does not imply cold:** a cool / FLR reset asserts primary (and hence core) but leaves the **cold domain untouched**, so the FLR / isolate programming that initiated a cool-reset flow deliberately survives it (§2.1, §2.4, §9). This separation of “wiped on warm reset” vs. “held across warm reset” is the entire reason the hierarchy exists.
 - **Implies:** core (the primary term gates `rst_core_n`).
 - **Outputs driven:** `rst_primary_ref_clk_no`, `rst_primary_smc_clk_no` (the register-block reset), `rst_primary_periph_clk_no` — i.e. distributed, synchronised, to the reference, SMC, and peripheral clock domains.

2.6.3 Core reset — SMC core/compute reset (innermost)

`rst_core_n` = `wdt_reset_n` & `rst_primary_n` & `fuse_reset_n` `rst_core_int_n` = `core_reset_n_ovrd` ? `core_reset_n_val` : `rst_core_n` where `fuse_reset_n` = `fuse_reset_ni` (JTAG-overrideable via `fuse_reset_n_ovrd`).

Core reset is the **innermost** member of the nested tree and resets the SMC **core / compute logic** without disturbing the chip-level register infrastructure. It has the **most trigger sources** because it folds the watchdog and fuse paths on top of the primary tree. It is asserted by **any** of:

1. anything that asserts **primary** (cold, incoming cool, FLR cool), **or**
 2. a **watchdog event** (`wdt_reset_n` low — see §2.6.4), **or**
 3. a **fuse reset** (`fuse_reset_ni` low — asserted while fuse sensing is in progress / not yet done), **or**
 4. a **JTAG core override** (`core_reset_n_ovrd`).
- **Scope / meaning:** holds the SMC core in reset. In this model core reset **does not clear any firmware registers** — the core domain holds compute state, not the register file — so its only observable effect is the `rst_core_smc_clk_no` output level.

- **Independently overridable:** JTAG can force core (`rst_core_int_n`) independently of the derived value, e.g. to hold the core during bring-up while the rest of the SMC runs.
- **Outputs driven:** `rst_core_smc_clk_no`.

2.6.4 WDT reset — watchdog reset (sibling)

$$\text{wdt_reset_n} = \text{rst_ext_wdt_ni} \& \sim \text{smc_wdt_second_timeout_i} \quad \text{rst_wdt_n} = \text{wdt_reset_n}$$

WDT reset is the watchdog's **dedicated reset output**. Unlike the other three it is **not** a level of the nested `cold` $\supset$ `primary` $\supset$ `core` tree — it is a sibling. It is asserted by **either**:

1. the **external watchdog reset pin** `rst_ext_wdt_ni` going low, **or**
 2. the **SMC watchdog second timeout** `smc_wdt_second_timeout_i` going high.
- **Relationship to core:** the same `wdt_reset_n` term also appears in the `rst_core_n` equation, so a watchdog event forces **core** as well — but it does **not** assert primary or cold. A watchdog timeout therefore resets the core (and raises the WDT output) while leaving the register file and FLR / isolate programming intact.
 - **First vs. second timeout:** only the **second** timeout participates in reset derivation. `smc_wdt_first_timeout_i` is accepted for interface fidelity (it is typically an early-warning interrupt) but has **no effect** on any reset (matches RTL, §5).
 - **Outputs driven:** `rst_wdt_smc_clk_no`.

2.6.5 Summary

Reset	Active-low signal	Asserted by	Forces (implies)	Registers cleared
Cold	<code>stable_cold_rst_n</code>	<code>!powergood_i</code> · <code>!rst_cold_ni</code> · JTAG cold	primary + core	cold-domain and primary-domain (§9)
Primary	<code>rst_primary_n</code>	<code>cold</code> · incoming cool (<code>!rst_cool_ni</code>) · FLR cool pulse · JTAG cool	core	primary-domain group (§9)
Core	<code>rst_core_int_n</code>	primary · WDT event · fuse (<code>!fuse_reset_ni</code>) · JTAG core	—	none (core holds compute state, not the reg file)
WDT	<code>rst_wdt_n</code>	<code>!rst_ext_wdt_ni</code> · <code>smc_wdt_second_timeout_i</code>	core (via shared term)	none

3. Address window

Property	Value
Access size	32-bit only, naturally aligned
Window size	<code>0x100</code> (8-bit decoded address; <code>RESET_UNIT_REG_SIZE = 0xCC</code>)
SMC base address	<code>0xC000_2000</code> (<code>smc_top.rd1</code> ; offsets are window-relative)
Endianness	Little-endian

4. Register map (from `reset_unit.rdl`)

Offset	Name	SW	Reset	Description
0x20	SS_CONFIG	rw	0x00000000	Per-SS configuration; locked bits (per <code>SS_CONFIG_LOCK</code>) are write-protected. Drives <code>ss_config_o</code> .
0x24	SS_CONFIG_LOCK	rw	0x00000000	woset : write-1-to-set sticky lock for <code>SS_CONFIG</code> .
0x40	SS_COLD_RESET_N	rw	0x00000000	Per-SS active-low cold reset; locked bits (per <code>SS_COLD_RESET_LOCK</code>) write-protected.
0x44	SS_WARM_RESET_N	rw	0xFFFFFFFF	Per-SS active-low warm reset.
0x48	SS_CONFIG_HOLD	rw	0x00000000	Preserve boot config across warm reset.
0x4C	SS_SRAM_HOLD	rw	0x00000000	Preserve SRAM/MBIST state across warm reset.
0x50	SS_CRITICAL_HOLD	rw	0x00000000	Preserve security/RAS/debug across warm reset.
0x54	SS_DEBUG_HOLD	rw	0x00000000	Preserve debug registers across warm reset.
0x60	SS_RESET_COMPLETE	r	0x00000000	Per-SS reset-complete status (driven by <code>ss_reset_complete_i</code>).
0x70	SS_COLD_RESET_LOCK	rw	0x00000000	woset : write-1-to-set sticky lock for <code>SS_COLD_RESET_N</code> .
0x80	SS_FORCE_T0_REF_CLK	rw	0x00000000	Force subsystem clock to ref clk during reset.
0x90	STRAPS_LO	r	(HW)	Captured straps [31:0] (<code>captured_straps_i[31:0]</code>).
0x94	STRAPS_HI	r	(HW)	Captured straps [63:32] (<code>captured_straps_i[63:32]</code>).
0xA8	SYNC_REG	rw	0x00000000	bit0: global sync IRQ → <code>sync_irq_o</code> .
0xB0	ISOLATE_REQ_REG	rw	0x00000000	SW per-SS isolate request.
0xB4	ISOLATE_REQ_PINEN_REG	rw	0x00000000	Enable external pin → isolate.
0xB8	ISOLATE_REQ_SMC_REG	rw	0x00000000	bit0: HW-set on FLR rising edge; any SW write clears it.
0xBC	ISOLATE_REQ_SMCEN_REG	rw	0x00000000	Enable FLR latch → isolate.
0xC0	ISOLATE_REQ_VIS	r	(HW)	bit0 = isolate pin, bit4 = <code>rst_cool_ni</code> , bit8 = <code>rst_cool_no</code> .
0xC4	ISOLATE_REQ_FLR_COUNTER_VALUE	rw	0x00000000	Cycles after FLR before <code>rst_cool_no</code> asserts.

Offset	Name	SW	Reset	Description
0xC8	ISOLATE_REQ_FLR_RESET_COUNTER_VALUE	rw	0x00000000	rst_cool_no assertion duration (cycles). 0 ⇒ FLR pulse suppressed .

Holes inside the window are **RAZ/WI**.

5. Reset-derivation contract

All resets are active-low. With JTAG overrides cleared:

```

powergood_stable = powergood_i
fuse_reset_n     = fuse_reset_ni
cold_reset_n     = rst_cold_ni
stable_cold_rst_n = powergood_stable & cold_reset_n
stable_cool_rst_n = rst_cool_ni
wdt_reset_n      = rst_ext_wdt_ni & ~smc_wdt_second_timeout_i
rst_primary_n    = stable_cold_rst_n & stable_cool_rst_n & cool_from_flr_n
rst_core_n       = wdt_reset_n & rst_primary_n & fuse_reset_n
rst_wdt_n        = wdt_reset_n

```

Synchronised, zero-delay, to the output ports:

Output	Equals
powergood_stable_o	powergood_stable
rst_cold_stable_ref_clk_no	stable_cold_rst_n
rst_cold_stable_smc_clk_no	stable_cold_rst_n
rst_primary_ref_clk_no	rst_primary_n
rst_primary_smc_clk_no	rst_primary_n
rst_primary_periph_clk_no	rst_primary_n
rst_core_smc_clk_no	rst_core_int_n
rst_wdt_smc_clk_no	rst_wdt_n
rst_cool_no	cool_from_flr_n (post-override)

smc_wdt_first_timeout_i is accepted for interface fidelity but does **not** participate in reset derivation (matches RTL).

6. JTAG overrides (set_jtag_ctrl)

Mirrors smc_pkg::jtag_smc_reset_ctrl_t. When an *_ovrd flag is set, the corresponding reset is forced to its *_val:

Override	Forces
<code>fuse_reset_n_ovrd</code>	<code>fuse_reset_n ← fuse_reset_n_val</code>
<code>cold_reset_n_ovrd</code>	<code>cold_reset_n ← cold_reset_n_val</code>
<code>cool_reset_n_ovrd</code>	<code>cool_from_flr_n / rst_cool_no ← cool_reset_n_val</code>
<code>core_reset_n_ovrd</code>	<code>rst_core_int_n ← core_reset_n_val</code>
<code>ss_cold_reset_n_ovrd[i]</code>	<code>ss_reset_ctrl_o[i].cold_reset_n ← ss_cold_reset_n_val[i]</code>
<code>ss_warm_reset_n_ovrd[i]</code>	<code>ss_reset_ctrl_o[i].warm_reset_n ← ss_warm_reset_n_val[i]</code>

7. Per-subsystem reset-control bundle (`reset_ctrl_t`)

For each subsystem `i ∈ [0, num_subsystems)`:

Field	Source
<code>cold_reset_n</code>	<code>SS_COLD_RESET_N[i]</code> (JTAG-overridable)
<code>warm_reset_n</code>	<code>SS_WARM_RESET_N[i]</code> (JTAG-overridable)
<code>config_state_hold</code>	<code>SS_CONFIG_HOLD[i]</code>
<code>critical_signal_hold</code>	<code>SS_CRITICAL_HOLD[i]</code>
<code>sram_hold</code>	<code>SS_SRAM_HOLD[i]</code>
<code>debug_hold</code>	<code>SS_DEBUG_HOLD[i]</code>
<code>force_to_ref_clk_n</code>	<code>SS_FORCE_TO_REF_CLK[i]</code>

8. FLR cool-reset flow

On a rising edge of `cfg_flr_pf_active_i`:

1. `ISOLATE_REQ_SMC_REG.bit0` is set (HW). Software clears it by writing the register.
2. If `ISOLATE_REQ_FLR_RESET_COUNTER_VALUE != 0`, `rst_cool_no` is driven low after `ISOLATE_REQ_FLR_COUNTER_VALUE × ref_clk_period_ns` and released after a further `ISOLATE_REQ_FLR_RESET_COUNTER_VALUE × ref_clk_period_ns`. A zero reset-counter value suppresses the pulse entirely.

```
skip_mem_repair_o = isolate_req_pin_i | ISOLATE_REQ_SMC_REG.bit0.
```

```
isolate_req_o[i] = ISOLATE_REQ_REG[i] | (ISOLATE_REQ_PINEN_REG[i] & isolate_req_pin_i) |  
(ISOLATE_REQ_SMCEN_REG[i] & ISOLATE_REQ_SMC_REG.bit0).
```

9. Reset domains for register state

Domain	Trigger (low)	Registers cleared
Primary (<code>rst_primary_smc_clk_no</code>)	cold or cool/FLR reset	<code>SS_CONFIG</code> , <code>SS_CONFIG_LOCK</code> , <code>SS_COLD_RESET_N</code> , <code>SS_WARM_RESET_N</code> → <code>0xFFFFFFFF</code> , <code>SS*_HOLD</code> , <code>SS_COLD_RESET_LOCK</code> , <code>SS_FORCE_T0_REF_CLK</code> , <code>SYNC_REG</code>
Cold (<code>rst_cold_stable_smc_clk_no</code>)	cold reset only	<code>ISOLATE_REQ_REG</code> , <code>ISOLATE_REQ_SMC_REG</code> , <code>ISOLATE_REQ_SMCEN_REG</code> , FLR counters; <code>ISOLATE_REQ_PINEN_REG</code> only when the isolate pin is low

Because the cold domain is **not** asserted by a cool/FLR reset, the FLR / isolate registers retain their values across a cool reset.

10. TLM-2.0 error taxonomy

Condition	Response
Access width $\neq$ 4 bytes	<code>TLM_BURST_ERROR_RESPONSE</code>
Misaligned (addr & <code>0x3</code> $\neq$ 0)	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>streaming_width</code> $\neq$ <code>data_length</code>	<code>TLM_BURST_ERROR_RESPONSE</code>
Byte-enable present	<code>TLM_BYTE_ENABLE_ERROR_RESPONSE</code>
Address $\geq$ <code>WINDOW_SIZE</code>	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
Command not READ/WRITE	<code>TLM_COMMAND_ERROR_RESPONSE</code>
Otherwise	<code>TLM_OK_RESPONSE</code>

DMI is never granted (register accesses have side effects). `transport_dbg` provides side-effect-free back-door access (no FLR latch / counter effects on write decode beyond the register value).

11. Timing

A configurable annotated delay (`access_delay_ns` , default 2 ns) is added to the `b_transport` delay argument for every register access. FLR pulse timing uses `ref_clk_period_ns` (default 10 ns). All reset derivation is otherwise combinational (zero-delay) in the LT model.